思灵机械臂 C++ 封装层
架构与实现方案

LancetRobotArm API Design Document

基于思灵机器人工业版 C API V2.19.0

Qt 5.12.10 | 医疗行业 | 安全优先

Lancet Medical Robotics
2026 年 4 月

目录

1. 项目概述
2. 文件结构
3. 类型定义（lancetrobottypes.h）
4. 状态机（lancetrobotstatemachine）
5. 医疗安全防护层（lancetrobotsafety）
6. 主类 LancetRobotArm（lancetrobotarm）
7. 路径规划辅助类（lancetrobotpath）
8. 单位转换机制
9. 线程安全模型
10. 随动功能扩展预留
11. 典型用法
12. 验证清单

1. 项目概述

本文档描述将思灵机器人工业版 C/C++ API（V2.19.0）封装为基于 Qt 5.12.10 的现代 C++ 类库的架构设计与实现方案。

1.1 设计目标

- 将约 200 个 C 函数封装为结构清晰的面向对象接口
- 状态机驱动——机械臂作为单一角色，运动冲突自动拒绝
- 医疗安全优先——独立安全层，指令预校验 + 运行时监控 + 紧急停止
- 自动单位转换——外部 mm/度，内部 m/rad，边界统一处理
- 异步运动 + waitForIdle() 阻塞等待模式
- PIMPL 隐藏 C API 依赖，公共头文件零污染
- 为后期随动功能预留集成接口（完全解耦）

1.2 技术栈

- 语言：C++11（兼容 Qt 5.12.10 的 MSVC 编译器）
- 框架：Qt 5.12.10 Core 模块
- 底层：思灵机器人 DianaApi.h C 接口库
- 目标平台：Windows 10

1.3 核心约束

约束	决策	原因
运动冲突	拒绝新指令，返回错误码	状态可预测，医疗场景不容许隐式行为
同步方式	异步 + waitForIdle()	灵活，运动中可插入监控逻辑
错误处理	返回 int 错误码	与底层 C API 一致，无异常开销
单位系统	外部 mm/度，内部 m/rad	医疗场景直觉友好，底层精确

2. 文件结构

```
src/robot/
├── lancetrobottypes.h          # 公共类型、枚举、常量、数据结构
├── lancetrobotstatemachine.h    # 状态机声明
├── lancetrobotstatemachine.cpp  # 状态机实现（转换表）
├── lancetrobotsafety.h         # 医疗安全防护层声明
├── lancetrobotsafety.cpp        # 安全防护层实现
├── lancetrobotarm.h            # 主类声明（QObject，PIMPL）
├── lancetrobotarm.cpp           # 主类实现（C API 调用、回调分发）
└── lancetrobotpath.h           # 路径 RAII 辅助类（header-only）
```

所有文件以 lancet 前缀命名，遵循项目组织规范。

2.1 依赖关系

```
lancetrobottypes.h        ← 被所有其他文件引用
lancetrobotstatemachine.h  ← lancetrobotarm.cpp 内部使用
lancetrobotsafety.h       ← lancetrobotarm.cpp 内部使用
lancetrobotarm.h          ← lancetrobotpath.h 引用（友元）
DianaApi.h                ← 仅 lancetrobotarm.cpp 引用（PIMPL 隔离）
```

3. 类型定义（lancetrobottypes.h）

3.1 基础类型别名

类型	底层	说明
LancetJoints	QVector<double>	关节数组，长度 = jointCount()
LancetPose	QVector<double>	6 元素 [x,y,z,rx,ry,rz]
LancetMatrix4x4	QVector<double>	16 元素，4×4 列优先齐次矩阵
LancetJacobian	QVector<double>	6 × jointCount 雅可比矩阵

3.2 单位配置 LancetUnitConfig

外部接口的单位系统配置。所有公共方法的位置、速度、加速度参数均使用此单位，内部自动转换。

字段	类型	默认值	说明
length	LengthUnit	Millimeter	长度单位
angle	AngleUnit	Degree	角度单位

内置转换因子方法：lengthToInternal(), lengthToExternal(), angleToInternal(), angleToExternal()。

3.3 状态枚举 RobotState

枚举值	含义
Disconnected	未建立网络连接
Idle	已连接且空闲，可接受运动指令
Moving	正在执行点到点或路径运动
SpeedControl	速度控制模式（speedJ / speedL）
ServoControl	伺服控制模式（servoJ / servoL 高频）
FreeDriving	拖动示教模式
ForceMode	力控模式
ImpedanceMode	阻抗控制模式
Paused	运动暂停
Error	错误状态，需 clearError() 恢复
RescueMode	救援模式

3.4 运动参数 LancetMotionParams

字段	类型	说明
velocity	double	运动速度（外部单位/s）
acceleration	double	加速度（外部单位/s²）
zvOrder	int	输入整形阶次（0=禁用）
zvFrequency	double	输入整形频率 Hz
zvDamping	double	输入整形阻尼比 0~1
avoidSingular	bool	奇异值回避（moveL 有效）

3.5 安全配置 LancetSafetyConfig

医疗级安全参数。指令预校验限值和工作空间边界使用外部单位（mm/度），力/力矩恒为 N / N·m。

字段	默认值	说明
maxJointVelocity	57.3 度/s	关节最大速度（≈1 rad/s）
maxCartVelocity	200 mm/s	TCP 最大平移速度
maxCartRotVelocity	28.6 度/s	TCP 最大旋转速度
maxJointAcc	114.6 度/s^2	关节最大加速度
maxCartAcc	1000 mm/s^2	TCP 最大平移加速度
maxTcpForceN	50 N	TCP 外力上限（0=不监控）
maxJointTorqueNm	空	各关节力矩上限（空=不监控）
enableWorkspace	false	是否启用工作空间限制
heartbeatTimeoutMs	2000 ms	心跳超时→急停

3.6 错误码（LancetRobotError 命名空间）

常量	值	含义
Success	0	成功
Timeout	-1	操作超时
InvalidParam	-2	参数无效
SafetyRejected	-3	安全层拒绝
JointCountMismatch	-4	关节数不匹配
NotConnected	-1013	未连接
CallingConflict	-2201	运动指令冲突
CollisionDetected	-2202	碰撞检测触发
FreeDrivingBusy	-2210	拖动模式中
FatalError	-2307	致命错误

4. 状态机（LancetRobotStateMachine）

独立非 QObject 类，由 LancetRobotArm 内部持有。自身不持有锁，调用方在转换前获取 QMutex。

4.1 事件枚举

```
Connected, Disconnected, MoveIssued, SpeedIssued, ServoIssued,
FreeDrivingEnabled, FreeDrivingDisabled, ForceModeEntered, ForceModeLeft,
ImpedanceModeEntered, ImpedanceModeLeft, MotionCompleted, Paused, Resumed,
Stopped, ErrorOccurred, ErrorCleared, RescueModeEntered, RescueModeLeft
```

4.2 状态转换表

当前状态	触发事件	目标状态
Disconnected	Connected	Idle
Idle	MoveIssued	Moving
Idle	SpeedIssued	SpeedControl
Idle	ServoIssued	ServoControl
Idle	FreeDrivingEnabled	FreeDriving
Idle	ForceModeEntered	ForceMode
Idle	ImpedanceModeEntered	ImpedanceMode
Idle	RescueModeEntered	RescueMode
Moving	MotionCompleted	Idle
Moving	Stopped	Idle
Moving	Paused	Paused
Paused	Resumed	Moving
Paused	Stopped	Idle
SpeedControl	Stopped	Idle
ServoControl	Stopped	Idle
FreeDriving	FreeDrivingDisabled	Idle
ForceMode	ForceModeLeft	Idle
ImpedanceMode	ImpedanceModeLeft	Idle
Error	ErrorCleared	Idle
RescueMode	RescueModeLeft	Idle
任何已连接	ErrorOccurred	Error
任何状态	Disconnected	Disconnected

4.3 状态转换图

```
                ┌──────────────────────────────┐
                │   Disconnected   │
                └──────────────────────────────┘
                 Connected │
                      ▼
  ┌──────────────────┐ MoveIssued ┌──────────────────┐ FreeDrive ┌──────────────────┐
  │  Moving  │ ──────────────────────│  Idle  │──────────────────────  │ FreeDriving │
  └──────────┬──────┬──────┘          └──────┬──────┬──────┬──────┘          └──────────────────┘
Paused │   │ Stopped   Speed │   │   │ Servo
   ▼ │          ▼ │ ▼
  ┌──────────────┐       ┌──────────────┐ │ ┌──────────────┐    ┌──────────────────┐
  │ Paused │       │ Speed │   │ │ Servo │    │ ForceMode │
  └──────────────┘         │ Ctrl  │   │ │ Ctrl  │    └──────────────────┘
            └──────────────┘ │ └──────────────┘    ┌──────────────────┐
                  │         │ Impedance │
   ┌──────────────────┐          │          └──────────────────┘
   │  Error   │      ──────── ErrorOccurred（from any connected）
   └──────────────────┘
```

4.4 冲突拒绝规则

运动指令（move/speed/servo）仅在 Idle 状态接受。例外：SpeedControl 下可更新 speedX，ServoControl 下可更新 servoX。

当前状态	返回错误码	说明
Disconnected	-1013 NotConnected	未连接
Moving / SpeedControl / ServoControl	-2201 CallingConflict	运动冲突
FreeDriving	-2210 FreeDrivingBusy	拖动模式中
Error	-2307 FatalError	需先 clearError()
其他非 Idle	-2201 CallingConflict	通用冲突

4.5 运动完成检测

通过 C API 状态回调的 jointAngularVel 监测：连续 3 帧所有关节角速度绝对值 < 0.001 rad/s 时判定运动完成，状态机从 Moving 转换到 Idle，并通过 QWaitCondition

唤醒 waitForIdle()。

5. 医疗安全防护层（LancetRobotSafety）

独立 QObject 类，由 LancetRobotArm 内部持有。提供两层安全保障：

5.1 指令预校验（Pre-dispatch Validation）

每个运动方法内部在调用 C API 前，先通过安全层校验参数合法性：

- validateJointMotion(vel, acc) — 关节速度 / 加速度不超过硬上限
- validateCartMotion(vel, acc) — 笛卡尔平移速度 / 加速度不超过硬上限
- validatePoseInWorkspace(pose) — TCP 目标位置在工作空间边界内

校验不通过 → 返回 SafetyRejected (-3) 错误码 + qCWarning 审计日志。

5.2 运行时监控（Runtime Monitoring）

每帧状态回调中调用 checkRuntimeStatus()，检查：

监控项	触发条件	响应
碰撞检测	bCollision == true	信号 + emergencyStop()
TCP 外力	tcpExternalForce > maxTcpForceN	信号 + emergencyStop()
关节力矩	|jointTorque[i]| > maxJointTorqueNm[i]	信号 + emergencyStop()
工作空间	tcpPose[x/y/z] 越界	信号 + emergencyStop()

5.3 紧急停止 emergencyStop()

绕过状态机正常转换表，直接：

- 调用 C API stop() 停止所有运动
- forceState(Error) 强制进入错误态
- QWaitCondition::wakeAll() 唤醒所有等待线程
- 发射 safetyViolation 信号通知上层

5.4 审计日志

```
Q_LOGGING_CATEGORY(lcRobotSafety, "lancet.robot.safety")


安全校验失败 → qCWarning(lcRobotSafety) << "..."
紧急停止触发 → qCCritical(lcRobotSafety) << "..."
状态转换     → qCInfo(lcRobotArm) << "State: X → Y"
```

5.5 安全配置单位处理

外部单位配置（mm/度）在 setSafetyConfig() 时一次性转换为内部单位（m/rad）缓存。运行时校验直接使用内部缓存，零额外开销。力和力矩（N / N·m）不做转换。

6. 主类 LancetRobotArm

继承 QObject，使用 Qt PIMPL 模式（Q_DECLARE_PRIVATE），DianaApi.h 仅在 .cpp 内引用。

6.1 连接管理

```
LancetRobotArm(const QString &ip, int jointCount = 7, QObject *parent = nullptr)
int  connectToRobot()        // initSrvV2 + 注册回调 + 配置心跳
void disconnectFromRobot()    // destroySrv + 注销实例
bool isConnected() const
int  setStatePushPeriod(int ms)
```

6.2 状态机查询

```
RobotState state() const
bool isIdle() const
bool isMoving() const
bool inError() const
int  waitForIdle(int timeoutMs = -1)  // 阻塞等待
int  clearError()                     // Error → Idle
```

6.3 点到点运动（require Idle → Moving）

```
// 关节空间
int moveJoint(const LancetJoints &target, const LancetMotionParams &params)
int moveJointToPose(const LancetPose &pose, const LancetMotionParams &params,
                    const QString &tcpName = QString())

// 笛卡尔直线
int moveLinear(const LancetJoints &target, const LancetMotionParams &params)
int moveLinearToPose(const LancetPose &pose, const LancetMotionParams &params,
                     const QString &tcpName = QString())

// 点动（连续，需 stop() 停止）
int jogJoint(int jointIndex, bool positive, double vel, double acc)
int jogTcp(MoveDirection dir, double vel, double acc, ...)
int rotateTcp(MoveDirection axis, double vel, double acc, ...)
```

6.4 速度 / 伺服控制

```
// 速度控制（Idle → SpeedControl，SpeedControl 下可更新）
int setJointVelocity(const LancetJoints &vel, double acc, double duration = 0.0)
int setCartVelocity(const LancetPose &vel, double transAcc, double rotAcc, ...)

// 伺服控制（Idle → ServoControl，ServoControl 下可更新）
int servoJoint(const LancetJoints &target)     // 高频 ≥500Hz
int servoCartesian(const LancetPose &target)
```

6.5 停止 / 暂停

```
int stop()          // 任何已连接状态 → Idle
int pause()         // Moving → Paused
int resume()        // Paused → Moving
int emergencyStop()  // 绕过状态机 → Error（安全层触发）
```

6.6 特殊模式

```
int enableFreeDriving(bool enable, FreeDriveMode mode = Normal)
int releaseBrake() / holdBrake()
int enterForceMode(const LancetForceParams &params)
int leaveForceMode(int exitMode = 0)
int setControlMode(ControlMode mode)  // Position / JointImpedance / CartImpedance
int enterRescueMode() / leaveRescueMode()
```

6.7 状态查询（线程安全，外部单位）

```
LancetRobotStatus status() const    // 完整快照
LancetJoints jointPositions() const // 关节角度（度）
LancetJoints jointVelocities() const
LancetJoints jointTorques() const   // N·m，不转换
LancetPose   tcpPose() const        // [mm, mm, mm, 度, 度, 度]
double       tcpExternalForce() const  // N，不转换
bool         isCollision() const
```

6.8 Qt 信号

信号	触发时机
stateChanged(new, old)	状态机转换时
errorOccurred(code, desc)	C API 错误回调
warningOccurred(code, desc)	C API 警告回调
statusUpdated(status)	每帧状态推送
motionCompleted()	运动完成（速度归零）
safetyViolation(desc, eStop)	安全违规时

7. 路径规划辅助类（LancetRobotPath）

RAII 包装类（非 QObject），构造时自动 createComplexPath，析构时自动 destroyComplexPath。支持链式调用。

```
LancetRobotPath path(arm);
path.addMoveLinear({300, 0, 400, 0, 180, 0}, 50.0, 300.0)
    .addMoveJoint(homeJoints, 30.0, 60.0)
    .addMoveCircular({350, 50, 400, 0, 180, 0}, 50.0, 300.0);

int ret = path.execute();  // require Idle → Moving
arm.waitForIdle(10000);
// path 析构时自动释放
```

7.1 接口列表

方法	说明
LancetRobotPath(arm)	构造，自动分配路径 ID
~LancetRobotPath()	析构，自动释放（未 execute 也安全）
addMoveLinear(pose, vel, acc)	添加直线段（外部单位）
addMoveJoint(joints, vel, acc)	添加关节段（外部单位）
addMoveCircular(pose, vel, acc)	添加圆弧段（外部单位）
execute()	执行路径，arm → Moving
pathId()	底层 C API 路径 ID

8. 单位转换机制

所有单位转换集中在 LancetRobotArmPrivate 的内联方法中，保证零遗漏、零重复：

8.1 转换方向

方向	长度	角度
外部 → 内部（调 C API 前）	mm × 0.001 → m	度 × π/180 → rad
内部 → 外部（返回给调用方）	m × 1000 → mm	rad × 180/π → 度

8.2 各数据类型的转换规则

数据类型	转换规则
LancetPose [0..2]	长度转换（mm m）
LancetPose [3..5]	角度转换（度 rad）
LancetJoints 全部元素	角度转换（度 rad）
MotionParams.velocity	关节运动用角度，笛卡尔运动用长度
MotionParams.acceleration	同上
力/力矩/电流 (N/Nm/A)	不转换

8.3 安全配置单位处理

LancetSafetyConfig 的限值使用外部单位填写。setSafetyConfig() 调用时，安全层内部一次性转换为 m/rad 缓存，后续运行时校验直接使用缓存，无额外开销。

9. 线程安全模型

C API 状态回调在独立线程中触发，与用户调用线程并发。本设计使用以下锁策略：

受保护资源	锁类型	用途
状态机转换	QMutex (m_stateMutex)	check + transition 原子性
缓存状态 m_cachedStatus	QReadWriteLock (m_statusLock)	回调写 / 用户读
waitForIdle 阻塞	QMutex + QWaitCondition	等待→唤醒
静态实例注册表	QMutex (s_registryMutex)	IP→实例映射
Qt 信号	Qt AutoConnection	自动跨线程投递

9.1 C API 回调分发

思灵 C API 的回调是全局函数指针。本设计使用静态 QMap<QString, Private*> 注册表，按 IP 地址分发到正确的 LancetRobotArmPrivate 实例：

```
namespace {
    QMutex s_registryMutex;
    QMap<QString, LancetRobotArmPrivate*> s_registry;
}

static void s_stateCallback(StrRobotStateInfo *info, const char *ip) {
    auto *d = findInstance(ip);
```

```
    if (d) d->onStateCallback(info);
}
```

10. 随动功能扩展预留

后期将增加机械臂随动功能（如术中导航跟踪）。该功能作为独立业务类实现，与本封装层完全解耦。本层已通过以下接口提供足够的集成能力：

随动需求	本层提供的接口
实时获取关节/TCP 状态	statusUpdated() 信号，周期由 setStatePushPeriod() 控制
高频位置追踪	servoJoint() / servoCartesian() — 支持 ≥1kHz 更新
判断是否可接管控制	state() / isIdle() 查询
紧急中断	stop() / emergencyStop()
单位一致	随动类使用同一 LancetUnitConfig

10.1 随动类集成骨架

```
class LancetFollowController : public QObject {
    Q_OBJECT
public:
    explicit LancetFollowController(LancetRobotArm *arm, QObject *parent = nullptr);

private slots:
    void onStatusUpdated(const LancetRobotStatus &status);

private:
    LancetRobotArm *m_arm;
    QTimer         m_controlTimer;  // 自有定时器驱动闭环
    // 连接 arm->statusUpdated() 获取实时状态
    // 用 arm->servoCartesian() 发送跟踪目标
};
```

11. 典型用法

11.1 基本运动

```
LancetRobotArm arm("192.168.1.10");

// 单位配置（默认即 mm + 度，可省略）
arm.setUnitConfig({LengthUnit::Millimeter, AngleUnit::Degree});

// 医疗安全配置
LancetSafetyConfig safety;
safety.maxCartVelocity   = 100.0;  // 100 mm/s
safety.maxTcpForceN      = 30.0;   // 30 N 急停
safety.enableWorkspace   = true;
safety.workspaceMin      = {-500, -500, 0, 0, 0, 0};
safety.workspaceMax      = { 500,  500, 600, 0, 0, 0};
arm.setSafetyConfig(safety);

QObject::connect(&arm, &LancetRobotArm::safetyViolation,
    [](const QString &desc, bool eStop) {
        qCritical() << "SAFETY:" << desc << (eStop ? "[E-STOP]" : "");
    });

arm.connectToRobot();
arm.waitForIdle();

LancetMotionParams p;
p.velocity     = 50.0;   // 50 mm/s
p.acceleration = 300.0;  // 300 mm/s²
arm.moveLinearToPose({300, 0, 400, 0, 180, 0}, p);
arm.waitForIdle(10000);

LancetPose pose = arm.tcpPose();  // 返回 [mm, mm, mm, 度, 度, 度]

arm.disconnectFromRobot();
```

11.2 路径规划

```
LancetRobotPath path(arm);
path.addMoveLinear({300, 0, 400, 0, 180, 0}, 50.0, 300.0)
    .addMoveLinear({350, 50, 400, 0, 180, 0}, 50.0, 300.0);
path.execute();
arm.waitForIdle(10000);
```

11.3 速度控制

```
LancetJoints slowVel = {5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0};  // 度/s
arm.setJointVelocity(slowVel, 10.0);  // 持续运动
QThread::sleep(2);
arm.stop();
arm.waitForIdle();
```

12. 验证清单

- 1. 连接 → connectToRobot() 成功 → state() == Idle
- 2. 运动中再发运动 → 返回 -2201 CallingConflict
- 3. waitForIdle() 在运动完成后可靠解除阻塞
- 4. 力超限 → safetyViolation 信号 + 自动 emergencyStop()
- 5. 工作空间越界 → 同上
- 6. 心跳超时 → Error 状态
- 7. Path RAII 析构安全（未 execute 不崩溃）
- 8. 所有信号跨线程安全投递（Qt AutoConnection）
- 9. 单位转换：moveLinearToPose({300,...}) → C API 接收 0.3 m
- 10. 单位转换：tcpPose() 返回 mm + 度
- 11. 安全限值：safety.maxCartVelocity=100 → 内部 0.1 m/s

— 文档结束 —